

Turning on encryption — Part A, step by step

Written 19 Aug 2026. For Alex. Everything in this document is reversible.

What you are about to do, in one paragraph

Trim's code already knows how to encrypt your financial data — that shipped to production today and is sitting there doing nothing, because the switch is off. These four steps turn it on **halfway**: your data gets encrypted into new columns, *and the readable original stays right beside it*. Nothing is deleted. If anything goes wrong at any point, you flip a setting back and Trim behaves exactly as it does now.

The step that actually deletes the readable copy is **migration 019**, and that is **not in this document**. It can wait months.

The honest risk picture

I checked the real database before writing this. Here is what is actually at stake.

Losing data: essentially impossible in these four steps. Every migration here only *adds* — add column if not exists, create index if not exists, create table if not exists. There is no drop, no delete, no update of existing rows anywhere in them. The backfill script only ever writes to the new `_enc` columns and is coded so it cannot touch a plaintext column.

Breaking the app: possible, and this is what to actually watch. Three ways:

What could break	When	How you'd know	Fix
Every write returns an error	You flip to dual before the key is set in Vercel	Adding a transaction fails in the app	Set the key, redeploy
New signups get no categories	After migration 018a	A brand-new account shows an empty category list	Covered by the test in Step 2
Every write to 10 tables is rejected	Only if the encryption_cutover row is deleted	Everything fails at once	One SQL line, in Rollback

The one genuinely unrecoverable thing is losing the key — and only *after* migration 019, months from now. Which is why Step 1 is about backing it up, not about generating it.

Your current data, as of now: 7 users, 267 rows across 10 tables. It is a small database, so every step here takes seconds.

Step 0 — the backup (already done)

I took a snapshot before writing this:

```
/Users/alexphan_bon/Trim-backups/trim-data-2026-08-19T05-15-52-043Z
```

267 rows across 10 tables, and the counts match what I read directly out of the database. To take a fresh one at any time:

```
cd "/Users/alexphan_bon/Vibe Coded Apps/Trim (Budgeting App)/server" && node scripts/sn
```

Be clear about what this is. It is a copy of your *data*, in JSON, saved outside the repo so it can never be committed. It does **not** include `auth.users` (Supabase Auth owns that and does not expose it) and it is not a `pg_dump`. For Part A that is plenty, because nothing here destroys anything. **Before migration 019 you will need a real backup that you have proven restores** — that is a Part B problem.

That folder holds every user's financial history in readable form. Keep it private, and delete it when the rollout is finished.

Step 1 — generate the key and back it up

This step is yours alone. I am not allowed to generate, read, or handle this key, and that rule is in `AGENTS.md` for a good reason: after migration 019, this key is the only thing that can read anyone's financial history. Run these in your own terminal, not through me.

1a. Generate it

```
openssl rand -base64 32
```

That prints one line — 44 characters ending in `=`. That is your key.

1b. Back it up in TWO places before you use it anywhere

Not one. Two. For example your password manager **and** a file in `~/Keys/`. Do this now, before pasting it anywhere else. Everything after this point assumes the key survives.

1c. Put it in your local `.env`

Open `server/.env` and add a line:

```
DATA_ENCRYPTION_KEY=<paste the key here>
```

No quotes, no spaces, no trailing newline inside the value. The code rejects a key with whitespace or quotes rather than silently using a different key — that check exists because `Buffer.from(..., 'base64')` quietly discards stray characters, which would encrypt everything under a key you don't have.

1d. Put it in Vercel

In the Vercel dashboard: *trim-api* -> **Settings** -> **Environment Variables** -> **Add New**

- Name: DATA_ENCRYPTION_KEY
- Value: the key
- Environment: **Production** only

Or in your own terminal, which prompts you for the value:

```
cd "/Users/alexphan_bon/Vibe Coded Apps/Trim (Budgeting App)/server" && vercel env add
```

1e. Redeploy so the key is actually loaded

Vercel env vars only apply to new deployments. Setting it changes nothing until:

```
cd "/Users/alexphan_bon/Vibe Coded Apps/Trim (Budgeting App)/server" && vercel deploy
```

CHECK — verify Step 1

Nothing should have changed at all — the phase is still off, so the key is set but unused.

```
curl -s https://trim-api-jade.vercel.app/api/health
```

Expect `{"status": "ok", ...}`. Then open Trim and add a test transaction. It should work exactly as before. **If it does, the key is loaded and harmless.**

To check your local key is well-formed without printing it:

```
cd "/Users/alexphan_bon/Vibe Coded Apps/Trim (Budgeting App)/server" && node -e "import
```

Expect KEY OK. It prints nothing about the key itself.

UNDO — Step 1

Delete the Vercel variable and redeploy. Nothing depends on it yet.

Step 2 — apply migrations 012, 018, 018a

In that order. Each one is safe to run twice — every statement is `if not exists`, so a run that fails halfway can simply be re-run.

How to run them

Option A — Supabase dashboard (recommended, you stay in control). Go to [the SQL Editor](#), open the file in your editor, copy the whole thing, paste, press Run. One file at a time.

- server/migrations/012_encryption_columns.sql
- server/migrations/018_encryption_text_columns.sql
- server/migrations/018a_encryption_write_barrier.sql

Option B — I apply them for you, one at a time via the Supabase connector, showing you the verification after each. Just say so.

What each one does

012 — adds `_enc` columns for the money: transaction amounts, budget limits, savings goal targets, contributions, your monthly limit, Ask Trim messages. Adds columns only.

018 — adds `_enc` columns for the *text*: descriptions, category names, goal names, notes, group names, subscription labels. Plus the “blind index” columns that let merchant memory keep working on encrypted data, and five indexes.

Two of those indexes are unique. If either fails with a duplicate-key error, **stop** — it means two rows collided (two default categories in one slot, or two merchant keys hashing the same). That needs looking at, not retrying.

018a — the one with a real behaviour change. It:

1. Creates the `encryption_cutover` table and installs “barrier” triggers on 10 tables. These do nothing until someone engages the flag, which happens only during migration 019.
2. **Stops the database’s signup trigger from creating the 12 default categories.** From here on, the app creates them instead (on `GET /api/me` and `GET /api/categories`), because the database has no encryption key and could not write encrypted names.

Point 2 is the only user-visible change in this whole step, and it is why the check below matters.

CHECK — verify Step 2

Run this in the SQL editor (or ask me to):

select

```
(select count(*) from information_schema.columns
  where table_schema='public' and column_name like '%\_enc') as enc_columns,
(select count(*) from information_schema.columns
  where table_schema='public' and column_name like '%hmac%') as blind_index_columns,
(select count(*) from pg_trigger where tname like '%encryption_cutover_guard') as ba
(select engaged from public.encryption_cutover where id) as barrier_engaged,
(select count(*) from public.transactions) as transactions,
(select count(*) from public.categories) as categories;
```

Expect: **17 enc_columns, 3 blind_index_columns, 10 barrier_triggers, barrier_engaged = false**, and transactions / categories **unchanged at 159 and 85**. Those last two are the point: the migrations added structure and touched no data.

Then the signup test — do not skip this one. Sign up a brand-new throwaway account in Trim and confirm it lands on a working dashboard **with the 12 default categories present**. That proves the seeding successfully moved from the database into the app. If the categories are missing, stop and tell me before going further.

UNDO — Step 2

You almost certainly won’t need to. The added columns are empty and ignored while the phase is off — leaving them costs nothing. If you want the barrier gone, the rollback block is at the top of `018a_encryption_write_barrier.sql`. To put category seeding back in the database, re-run the `handle_new_user()` function from `001_init.sql`.

Step 3 — switch to dual

This is the flip. From here, **every write saves both the readable value and the encrypted one.**

Do not do this before Step 1 and Step 2 are both verified. At dual the app writes to the `_enc` columns using the key — if the columns don't exist, or the key isn't there, every write fails.

Set it

Vercel dashboard: *trim-api* -> **Settings** -> **Environment Variables** -> **Add New**

- Name: ENCRYPTION_PHASE
- Value: dual
- Environment: **Production**

Then redeploy — again, the variable does nothing until you do:

```
cd "/Users/alexphan_bon/Vibe Coded Apps/Trim (Budgeting App)/server" && vercel deploy --
```

Type dual exactly. A typo like Dual is fine (it trims and lowercases) but dualwrite is not — the server **refuses to boot** on an unrecognised value rather than guessing. If the API goes down right after this step, that is the first thing to check.

CHECK — verify Step 3

```
curl -s https://trim-api-jade.vercel.app/api/health
```

Then **add a real transaction in the app** — something obvious like “runbook test, £1.23”. Then run this SQL:

```
select id, description is not null as has_plaintext,  
       description_enc is not null as has_ciphertext,  
       amount_enc is not null as has_amount_enc  
from public.transactions  
order by created_at desc limit 3;
```

The newest row must have **all three true**. That is dual-write working: readable value saved, encrypted value saved alongside. Older rows will still show `has_ciphertext = false` — Step 4 fixes those. Delete your test transaction in the app afterwards if you like.

UNDO — Step 3

Change ENCRYPTION_PHASE back to off (or delete the variable) and redeploy. Trim goes back to writing plaintext only, and the encrypted columns just sit there ignored. **This is the single most useful undo in the whole rollout** — it takes about a minute and reverses everything Step 3 and Step 4 did.

Step 4 — backfill the existing rows

Step 3 encrypts everything written *from now on*. This encrypts what's already there — your 159 transactions and the rest.

It runs from your laptop, not from Vercel, using the key in `server/.env`.

Dry run first

```
cd "/Users/alexphan_bon/Vibe Coded Apps/Trim (Budgeting App)/server" && node scripts/en
```

This writes nothing. Read the summary; it tells you how many rows it *would* touch.

Then for real

```
cd "/Users/alexphan_bon/Vibe Coded Apps/Trim (Budgeting App)/server" && node scripts/en
```

With 267 rows this takes seconds. It is safe to re-run — it skips rows already done, and if it aborts partway you just run it again.

The script does something worth knowing about: for every row it encrypts, it writes, then **reads the row back out of the database and decrypts what Postgres actually returned**, and compares that to the original. If anything doesn't match it stops immediately. It never logs amounts or descriptions, only row IDs.

CHECK — verify Step 4

```
cd "/Users/alexphan_bon/Vibe Coded Apps/Trim (Budgeting App)/server" && node scripts/ve
```

--sample means “diagnostic, not an authorisation” — it deliberately can't approve anything irreversible, which is right, because nothing irreversible is happening. With only 267 rows, 1000 covers all of them. It will print **INCOMPLETE** by design; what you're reading is whether it reports any *failures*.

Then confirm nothing is left behind:

```
select
  (select count(*) from public.transactions where description is not null and description_enc is null) as txns,
  (select count(*) from public.categories where name is not null and name_enc is null) as cats,
  (select count(*) from public.budgets where amount_limit is not null and amount_limit_enc is null) as budgets;
```

All three should be 0.

Finally, **open Trim and click around** — dashboard, transactions, budgets, a goal, Ask Trim. Everything should look completely normal, because it is: the app is still reading the plaintext columns. That is the whole point of dual.

UNDO — Step 4

Nothing to undo — it only filled in new columns. If you wanted to start over you could null the `_enc` columns and re-run, but there is no reason to.

You're done. What's true now

- Every piece of financial data in Trim exists **twice**: readable, and encrypted with a key that lives only in your backups and Vercel.
- Anyone looking at the Supabase dashboard still sees the readable copy — ***that's expected, and it's what makes this reversible.***
- The app behaves identically to before.
- You can undo the whole thing by setting `ENCRYPTION_PHASE=off` and redeploying.

What's left, and why it can wait

Part B — migration 019 deletes the readable columns. That is when the privacy goal is actually met, and it's the irreversible one. Before it runs:

1. The parked “cutover machinery” (the gate and the barrier) ***must get an independent code review***. It was written across four rounds and the last round was never reviewed by a second model, because Codex refused to look at this branch. That review is a hard condition, written into `CHAT_HANDOFF.md`.
2. You take a ***real*** backup — a `pg_dump`, restored somewhere and checked — not the JSON snapshot from Step 0.
3. The full 10-step sequence in `SECURITY.md` runs: disable the cron, engage the write barrier, run the gate until it exits 0, back up, drop, release, `ENCRYPTION_PHASE=enc`.

There is no deadline on any of that. Part A on its own is a real improvement, and it is where all the risk stops.

Rollback card — keep this handy

Problem	Fix
Writes failing after Step 3	<code>ENCRYPTION_PHASE -> off</code> in Vercel, redeploy
API won't start after Step 3	The phase value is misspelled. It must be exactly <code>off</code> , <code>duál</code> , or <code>enc</code>
“ <code>DATA_ENCRYPTION_KEY</code> is not set”	Key missing in Vercel, or set but not redeployed
“must be canonical base64”	The key has a space, quote or newline in it. Re-paste from your backup
Every write rejected, all tables	The barrier flag row is missing. Run: <code>insert into public.encrypted_cutover (id, engaged) values (true, false) on conflict (id) do nothing;</code>
New signups have no categories	Open <code>GET /api/me</code> (just load the app). If still empty, tell me — don't patch it live
Want everything back exactly as today	<code>ENCRYPTION_PHASE=off</code> , redeploy. The extra columns are inert

One thing that is never recoverable: losing `DATA_ENCRYPTION_KEY` after migration 019. Not

during Part A — during Part A the readable data is still there. But the habit starts now: two backups, offline, before you use it.